

Relatório de parada — Coding_Machine / DEVFACTORY

Data da parada: 29/09/2026, ~13:30 (America/São Paulo) **Autor do pedido:** Ed (edsauro) **Motivo:** parar toda a execução e congelar o estado, com o histórico, as fases, os artefatos e os arquivos (usados e gerados) em um documento que sirva de **base para um novo planejamento**. **Projeto:** `~/Code/Coding_Machine` · branch `main` · HEAD `c49edb4` (sincronizado com `origin/main`). **Estado final verificado:** 4 sprints executadas, **36/36 tarefas integradas**, **450 testes verdes**, nenhum processo do motor rodando. **Como ler:** todo número aqui foi medido no `state.db`, no git ou no disco — não vem de resumo de memória. Onde há divergência com outro documento do repositório, o relatório aponta (ver §11).

0. A parada, em concreto (o que está desligado e como religar)

o que	estado agora	como religar
Execução de cron do Hermes	<code>pausada</code> (<code>hermes pause</code> → sentinel <code>~/ .hermes/ESTOP</code>)	<code>hermes resume</code>
Cron <code>c9ff58c7925d</code> — vigia DEVFACTORY-003 (30 min)	<code>pausado</code>	<code>hermes cron resume c9ff58c7925d</code>
Cron <code>5eb37e814ff2</code> — lembrete do A/B (segunda 9h)	<code>pausado</code>	<code>hermes cron resume 5eb37e814ff2</code>
Cron <code>08885dbb91d1</code> — auditoria de recursos (ter/sáb 4h)	<code>pausado</code>	<code>hermes cron resume 08885dbb91d1</code>
Crons <code>329c5e1d03b4</code> , <code>aefcbe58a06c</code> , <code>de834c952bcd</code>	já concluídos/pausados antes	—
Processo do motor (<code>autodev run</code>)	<code>nenhum rodando</code>	<code>cd ~/Code/Coding_Machine && .venv/bin/python -m autodev --sprint <S> run --rodadas 5</code>
Killswitch do motor (<code>state.db</code>)	inativo (<code>STOP_SPRINT \ 0</code>)	<code>autodev start</code>

O vigia era o único agente que podia **religar o motor sozinho** (o prompt dele mandava retomar o sprint quando não houvesse rodada viva). Sem ele e sem a entrega de cron, a parada é estável: nada acorda sozinho.

Última ação automatizada antes da parada: o vigia rodou às 13:06 e fez o commit `c49edb4` — “Revisao de processo: licoes aprendidas + verificador de dono de arquivo” — que criou `LICOES-APRENDIDAS.md` e `.autodev/scripts/checar_dono_arquivo.py`. Foi essa revisão que motivou a decisão de parar.

1. O que é a máquina (para quem for replanejar sem contexto)

Uma fábrica de desenvolvimento autônomo: recebe uma especificação de sprint, quebra em tarefas (ou recebe o DAG pronto), executa cada tarefa com um agente de código dentro de um **worktree isolado**, roda testes determinísticos, submete a revisão por um **agente diferente** do que escreveu, corrige, re-testa e

integra num branch de sprint — **nunca em main** .

```
SPEC → PLAN → DAG DE TAREFAS → WORKTREE → AGENTE DE CÓDIGO → TESTE  
→ REVISÃO INDEPENDENTE → CORREÇÃO → RETESTE → INTEGRAÇÃO  
→ VALIDAÇÃO FINAL → RELATÓRIO
```

Três princípios que sustentam o desenho:

1. **Quem escreve não revisa** (Codex escreve / Hermes revisa, por exemplo) — auto-revisão não encontra o erro de quem escreveu.
2. **Cota de API não é falha**. Esgotar cota vira espera de recurso, com checkpoint e retomada por **continuação**, não reinício.
3. **Portão determinístico é piso, nunca aprovação**. O que aprova é o revisor; o portão só barra o que é objetivamente quebrado.

2. Histórico por fases

Datas e janelas vindas dos commits e do `state.db` (tabela `attempts`).

F0 — 25–26/09 · o motor nasce

- Sandbox `bubblewrap` , ciclo de vida do sprint, portões de integração, hook de commit, HAQ-001 (nenhuma credencial dentro do projeto: `~/.codex/auth.json` é montado **somente leitura**, nunca copiado).
- **DEVFACTORY-001** executada (27/09, 02:03 → 02:07, 15 chamadas). 15/15 tarefas — mas 14 delas foram marcadas depois como `origem='retroativo'` : a evidência foi reconstruída, não observada. **É um dos motivos de a 001 não estar no relatório de tentativas**.

F1 — 27/09 · os primeiros defeitos de motor

- D-07 (o modelo de dependências estava errado), D-08/D-09 (cota do AGY e colisão de “disk quota” no classificador), correção de privacidade (contexto de saúde fora do repo, com hook que barra).
- **Matriz do autor** (`3b94dc2`): escadas de implementação e revisão, revisor Hermes, `desbloquear` , HAQ-001 registrado.

F2 — 27/09 03:09 → 28/09 09:52 · o planejador (DEVFACTORY-002)

- 10 tarefas, **73 chamadas**, ~109k tokens. Objetivo: o motor passar a receber um pedido em texto livre e produzir spec + DAG executável, validado pelas mesmas regras da carga.
- D-16 (causa-raiz: o DAG mandava 8 de 10 tarefas escreverem **no mesmo arquivo de teste**), D-17, D-18, D-19 (preservação com evolução), D-20 (a espera de cota usa o reset informado pelo agente), D-21/D-22 (registro de decisões e revisão retroativa por LLM).
- Entregas de apoio: tela permanente de eventos (`9155289`), guarda de sobreposição confiável (writer lock em vez de `pgrep`), `autodev plan` na CLI, planejamento do verificador de planos.

F3 — 28/09 · correções e a segunda frente (DEVFACTORY-004)

- Revisão retroativa da sprint 2 → plano da 004 (4 tarefas). D-23: **o comando dos critérios é o que o runner resolve** — 12 tentativas queimadas por erro de plano, não de modelo. D-24/D-25: rodada única garantida pelo motor e portão de segurança com escopo no diff.
- Fusão das branches na `main` por soma de lados (292 verdes na época).

F4 — 28/09 14:19 → 19:43 · DEVFACTORY-004 executada

- 4 tarefas, **24 chamadas**, ~138k tokens. Correções de README/hierarquia, teste de documentação isolando a seção do Planejador, comando de teste que roda nesta máquina.
- Relatório de tentativas nasce (`c661837`): chamadas de API por pacote e por tentativa.

F5 — 28/09 22:58 → 29/09 08:51 · DEVFACTORY-003 (a maior)

- 7 tarefas, **27 chamadas**, ~1.541k tokens (a sprint que mais gastou, por causa das tentativas de contrato ruim e de uma espera de cota).
- Entregas: detecção de colisão de arquivo entre tarefas, `autodev prever` (impacto do plano antes de rodar, `plan_impacto.py`), **portão de aprovação humana do plano** e sua aceitação ponta a ponta.
- Em paralelo, no motor: P-09 (classe sem escalonamento não gasta degrau), P-10 (tokens por tentativa, lendo o rodapé do agente), P-12 (contador de escalonamento direcional), P-13 (a escada escala **só por modelo**), P-14 (`cota.py` lê o `resetsAt` da janela certa) e P-15 (estratégias de alocação gravadas por task, com o portão humano `WAITING_HUMAN`).
- Relatório de tentativas reorganizado em **4 painéis alinhados no eixo x**, abertos por modelo de tentativa, mais a seção de estratégias, auditoria de data-hora por pacote e a marcação de qual estratégia cada pacote está.

F6 — 29/09 (pós-09:00) · revisão e parada

- O vigia produz `LICOES-APRENDIDAS.md` : **75% dos findings de rejeição são defeito de contrato ou de plano; 3% são da implementação**. Conclusão que motivou a parada: escalar modelo num contrato quebrado paga caro para receber a mesma falha.
- Autor decide parar tudo → este documento.

3. Arquitetura do motor (código)

`autodev/` — **21 módulos, 6.281 linhas** (sem os testes):

módulo	linhas	papel
orchestrator.py	951	laço principal: ondas, tentativas, escalonamento, trabalho por task
state.py	1005	SQLite: tasks, attempts, eventos, métricas, killswitch, estratégia
cli.py	629	comandos: plan , run , status , desbloquear , prever , stop , start
config.py	424	carrega YAML: agentes, modelos (escadas), políticas, estratégias
planner.py	391	tipos do plano, validação de schema, colisão na mesma onda
agents.py	385	invocação dos agentes (Codex CLI, AGY, Hermes) com sandbox
report.py	347	relatório por sprint (snapshot do estado)
review.py	330	revisor independente (modelo e esforço vindos da config)
integration.py	267	merge de onda, portões build/testes/lint/segurança
retry.py	267	classificação de falha e decisão de escalonamento
testrunner.py	237	comando de teste declarado no critério + parsing
sandbox.py	234	bubblewrap , HOME isolado, montagem somente leitura
errors.py	201	taxonomia de falhas (REVIEW_FAILURE , CODEX_QUOTA , ...)
worktree.py	193	worktree por task, a partir das dependências
cota.py	141	lê resetsAt das janelas (5h e semanal) e calcula a espera
haq.py	126	perguntas de bloqueio humano (HAQ) e seu ciclo
killswitch.py	83	parada de projeto/sprint/task
plan_prompt.py	36	prompt do planejador
plan_impacto.py	28	impacto do plano antes de rodar

Testes: 34 arquivos, **5.792 linhas**, **450 verdes** (comando na §13).

Fluxo de uma tentativa: task → worktree (base nas dependências) → agente no sandbox → testes do critério → revisor independente → APPROVE/REQUEST_CHANGES com findings → escalonamento de degrau se reprovar → commit → integração por onda → portões → main só por merge manual (o motor nunca faz).

4. Artefatos de desenvolvimento (o que foi construído e onde vive)

4.1 Código e testes

- `autodev/*.py` (21 módulos) — o motor.
- `.autodev/tests/*.py` (34 arquivos) — a suíte.
- `.autodev/fixtures/` — fixtures (ex.: estrutura de README para o teste documental).
- `.autodev/scripts/` — ferramentas operacionais, não portões: `tela.py` (tela permanente de eventos), `estado_003.sh` (status + linha de Telegram), `rodada_em_andamento.py` (guarda de sobreposição), `verificar_plano.py` (colisão e critério vago), `checar_dono_arquivo.py` (**criado na revisão de 29/09**), `aceitacao_real.py`, `backfill_tokens.py`.
- `.autodev/config/` — `agents.yaml`, `models.yaml` (escadas e matriz), `policies.yaml` (retry, `classes_sem_escalamento`, `retry.estrategias` com a “3 degraus”).

4.2 Planejamento e decisão (por sprint)

Cada sprint tem `spec.md`, `plan.md` / `dag.json`, `sprint.yaml`, `decisions.md`, `SPRINT-REPORT.md`, `HAQ.md`, `evidence/` e `logs/`:

sprint	decisions.md	tasks	chamadas	tokens	resultado
DEVFACTORY-001	308 linhas	15	15	não medido	15/15 (14 retroativas)
DEVFACTORY-002	667 linhas	10	73	~109k	10/10
DEVFACTORY-003	44 linhas	7	27	~1.541k	7/7
DEVFACTORY-004	210 linhas	4	24	~138k	4/4

4.3 Documentos de topo

- `README.md` — visão, fluxo, por que não é “Codex num loop”.
- `Pendencias.md` (289 linhas) — o que está pendente, quem decide, caminhos.
- `LICOES-APRENDIDAS.md` — revisão de processo de 29/09 (ver §11: a tabela de sprints dele estava errada e foi corrigida com nota).
- `TesteAB-eficiencia-LLM.md` — plano do estudo A/B (PLANEJADO, não executado).
- `REPORT-TENTATIVAS-CODEX.md` + `report/` — o relatório visual e seus insumos.

4.4 Relatório de tentativas (o produto analítico)

- `report/gerar_relatorio_tentativas.py` — gerador (gráficos + Markdown da **mesma** agregação; o texto e o gráfico não podem divergir).
- `report/paineis-comparacao.png` — os 4 painéis no mesmo eixo x: chamadas, custo, tokens e complexidade (linhas/arquivos do commit + findings do revisor), abertos por modelo.

- `report/histograma-tentativas.png` , `distribuicao-tentativas.png` , `custo-por-pacote.png` — os gráficos detalhados.
- `report/dados.json` — a agregação crua, auditável.
- `report/relatorio-tentativas.pdf` (14 páginas, ~1 MB) + `.html` (autocontido) + `report/estilo-relatorio.css` .
- Escopo declarado: **sprints 003 e 004** (as únicas com registro de modelo por tentativa); 001 e 002 aparecem fora do recorte, com a razão no próprio PDF.

4.5 Skills do Hermes usadas/geradas para este trabalho

`llm-coding-efficiency` , `capability-ab-test` , `agent-cost-audit` , `autonomous-coding-loop` , `agent-bridge` , `coding-agent-sessions` , `relatorio-para-pdf` , `resource-auditor` , `spec-solucoes-dev` , `claude-handoff` , `subagent-fan-out` , `computer-use` , `hermes-agent` .

5. Arquivos USADOS (entradas)

arquivo / recurso	papel	observação de segurança
<code>.autodev/state.db</code>	fonte da verdade do motor (tasks, attempts, eventos, killswitch, estratégia)	1,4 MB; lido em modo somente leitura pelo relatório
<code>.autodev/sprints/<S>/sprint.yaml</code> , <code>dag.json</code> , <code>spec.md</code> , <code>plan.md</code>	definição do backlog e do grafo	entrada do orquestrador
<code>.autodev/config/*.yaml</code>	agentes, modelos/escadas, políticas e estratégias	mudanças aqui só valem na próxima largada do processo
<code>~/codex/auth.json</code>	credencial do Codex CLI	[nunca copiada] ; montada somente leitura no sandbox (<code>--ro-bind-try</code>)
CLI <code>agy</code> / <code>ask-agy</code>	agente de revisão da escada (Antigravity)	cota própria; indisponibilidade cai para a reserva
<code>~/hermes/.env</code>	chave do provider DeepSeek (revisor)	fora do repositório
gateway do Hermes (Telegram)	aviso de status e portão humano	<code>hermes send --to telegram</code>
<code>UV</code> + <code>venv</code> descartável	<code>matplotlib/pandoc/Chromium</code> para os gráficos	não se instala pacote de gráfico no <code>venv</code> do projeto
repositório do projeto	o alvo do trabalho de cada tarefa	o motor integra em branch de sprint; <code>main</code> só por merge manual

6. Arquivos GERADOS (saídas)

caminho	volume	o que é
.autodev/worktrees/	91 MB (28 worktrees)	um worktree por tarefa, com branch própria
.autodev/sprints/	6,1 MB	specs, planos, decisions, relatórios, 137 arquivos de evidência, 177 logs
.autodev/state.db	1,4 MB	banco de estado (cresce a cada tentativa)
.autodev/tests/	1,5 MB	suíte (34 arquivos, 450 testes)
.autodev/scripts/	116 KB	ferramentas operacionais
.autodev/fixtures/	208 KB	fixtures de teste
.autodev/config/	20 KB	configuração do motor
.autodev/ total	101 MB	inclui os worktrees
report/	4,1 MB	PDF (1,0 MB), HTML (1,0 MB), 8 PNGs, dados.json , CSS, gerador
.git/	17 MB	130 commits, 818 arquivos alterados acumulados, +141.724/-6.857 linhas
branches	44	branches de task + de integração por sprint
webhooks/crons	7 jobs	ver §0

7. Números que sustentam decisões

Produtividade (fonte: state.db + LICOES-APRENDIDAS §3)

sprint	chamadas	aprovadas	REVIEW_FAILURE	esperas de cota
001	15	15	0	0
002	73	43	17	5
003	27	7	18	1
004	24	4	7	2
total	139	69 (50%)	42 (30%)	8

Causa-raiz dos findings de rejeição (003/004, 147 findings)

causa raiz	findings	%
PLANO — dono de arquivo	41	28%
CONTRATO — fato volátil	33	22%
CONTRATO — critério ambíguo	29	20%
CONTRATO — comando/runner	7	5%
não classificado (precisa leitura manual)	32	22%
IMPLEMENTAÇÃO	5	3%

Economia de cota (Codex) — modelo de custo: **US\$ 20/mês = 400% de cota semanal**; 1% da janela de 5h ≈ 16.871 tokens; 1% da semanal ≈ 118.096 tokens; **1% da semanal ≈ US\$ 0,05** (≈ US\$ 0,42/Mtok).

Teste A/B (prévia medida, ainda não executado como estudo) — custo por chamada:

braço	US\$ por chamada
deepseek-flash/high	0,0027
sol/low	0,0066
luna/low	0,0105
terra/low	0,0106
deepseek-v4-pro/high	0,0133
astra/low	0,0201

Padrão fixado pelo autor a partir disso: **DeepSeek sempre no high** , **Codex sempre no low** — e sol/high reprovou (o high no Codex não comprou acerto).

Modelos efetivamente usados nas tentativas registradas:

modelo/esforço	chamadas
gpt-5.6-luna/low	47
gpt-5.6-sol/low	28
gpt-5.6-sol/medium (matriz antiga)	14
gpt-6-astra/low	13
gpt-5.6-terra/low	11
gpt-5.6-luna/medium (matriz antiga)	5
gpt-5.6-terra/medium (matriz antiga)	2
sem registro (sprint 001)	15

8. Decisões registradas (o que não se rediscute sem motivo)

id	decisão
HAQ-001	nenhuma credencial dentro do projeto; ~/ .codex/auth.json montado somente leitura
D-07	o modelo de dependências do motor estava errado; worktree nasce das dependências, integração a cada onda
D-08/D-09	cota do AGY não estaciona a task; “disk quota” entra no classificador
D-15	cota do agente secundário não para a task (regra “não parar”)
D-16	causa-raiz: o DAG mandava 8 de 10 tarefas escreverem no mesmo arquivo de teste
D-17	entrega vazia (pergunta de design em headless) e ruído de bytecode
D-18	worktree sempre na base atual; rodada em passadas que não param no tropeço
D-19	preservação com evolução (a regra absoluta do D-16 tornava uma tarefa impossível)
D-20	a espera de cota usa o reset informado pelo agente (a política vira teto)
D-21/D-22	registro das decisões; fechamento e revisão retroativa por LLM
D-23	o comando dos critérios é o que o runner resolve (12 tentativas queimadas por erro de plano)
D-24	rodada única garantida pelo motor (writer lock, não pgrep)
D-25	portão de segurança com escopo no diff
P-12/P-13	contador de escalonamento é direcional; a escada escala só por modelo (4º degrau astra/low)
P-14	a espera de cota mira a janela certa (resetsAt), não um prazo fixo de 5h10m
P-15	estratégia de alocação gravada por task e imutável na largada; “3 degraus” com parada para decisão do autor

9. Pendências abertas no momento da parada

id	pendência	estado
P-11	estudo A/B de eficiência de LLM com N≥3	plano escrito (TesteAB-eficiencia-LLM.md), prévia medida, nunca executado
P-14	espera de cota pela janela certa	implementada (cota.py + testes)
P-15	estratégias de alocação + portão humano	implementada ; a “3 degraus” nunca rodou em produção
—	próxima sprint (005)	não planejada ; fila vazia

Também aberto, do `LICOES-APRENDIDAS.md` (§9, decisões do autor): refazer, revisar ou refazer do zero; e a avaliação sobre comprar API do Claude/Anthropic.

10. O que importa para o próximo planejamento

1. **O gargalo é o contrato, não o modelo.** 75% dos findings são de contrato/plano; 3% da implementação. Qualquer plano que comece por “modelo melhor” ataca 3% do problema.
2. **Os verificadores existem e não são portões.** `verificar_plano.py` e `checar_dono_arquivo.py` são scripts manuais. Hoje há 4 portões **por task** (build, testes, lint, segurança) na integração — e **nenhum portão antes de gastar cota**.
3. **O planejador construído na sprint 002 nunca rodou de verdade.** O laço ainda consome DAG escrito à mão.
4. **Ativos levantados nunca usados** (28 repositórios analisados, BMAD instalado, `superpowers` instalado como skill do Codex): zero menção em specs, decisions ou DAGs.
5. **Custo por chamada é conhecido; a decisão de escalonamento pode ser econômica.** Com a régua medida (§7), a pergunta “a 4ª/5ª posição da escada se paga?” passa a ter resposta por pacote, não por impressão.
6. **O que estreia na próxima sprint já tem mecanismo pronto e não exercitado:** estratégia por task, portão humano (`WAITING_HUMAN`), espera de cota pela janela certa, linha de status no Telegram por pacote.
7. **O que eu faria primeiro, se o replanejamento for meu:** transformar `checar_dono_arquivo.py` e `verificar_plano.py` em **portão de plano** (antes da primeira chamada de agente), porque é o único item que mexe no gargalo dos 75%.

11. Divergências e armadilhas conhecidas (para não repetir erro de leitura)

1. `LICOES-APRENDIDAS.md` §2 estava errado (“002 = 1/10”, “004 em andamento”). O `state.db` mostra 002 = 10/10 e 004 = 4/4. A nota de correção foi adicionada no topo do arquivo em 29/09; as causas-raiz (§3/§4) seguem válidas.
2. **O relatório de tentativas cobre só as sprints 003 e 004.** A 001 não tem registro de modelo por tentativa (evidência retroativa) e a 002 rodou parte antes do registro. Não é omissão: está declarado no próprio PDF.
3. **A “3 degraus” não tem execução real.** O 003/P07 foi **gravado** nela, mas executou a escada antiga (a troca de código não valeu para a rodada em curso). Está marcado com `*` nos gráficos e na tabela.
4. **Mudança de config não vale no processo em curso:** a matriz de modelos é lida na largada.

5. **Duas numerações de “tentativa”:** `attempt` (sequência do banco, sempre `max+1`) e o **degrau** (contador da task, que reinicia em rearme/reabertura). Ler uma como a outra produz conclusão errada.
6. **main nunca é tocada pelo motor.** 44 branches vivas; a integração de sprint é um merge manual, feito depois com a suíte verde.

12. Como reproduzir (comandos)

```
# suíte do motor (esperado: 450 verdes)
cd ~/Code/Coding_Machine && .venv/bin/python -m pytest .autodev/tests -q

# status de um sprint
.venv/bin/python -m autodev --sprint DEVFACTORY-003 status

# rodar/retomar um sprint
.venv/bin/python -m autodev --sprint <SPRINT> run --rodadas 5

# relatório de tentativas (venv descartável com matplotlib)
~/hermes/cache/scratch/venv_report/bin/python report/gerar_relatorio_tentativas.py
pandoc REPORT-TENTATIVAS-CODEX.md -o report/relatorio-tentativas.html --standalone \
  --embed-resources --resource-path=./report --css=report/estilo-relatorio.css \
  --metadata lang=pt-BR
chromium --headless=new --disable-gpu --no-sandbox --user-data-dir=/tmp/chrome-pdf \
  --virtual-time-budget=30000 --no-pdf-header-footer \
  --print-to-pdf="$PWD/report/relatorio-tentativas.pdf" \
  "file://$PWD/report/relatorio-tentativas.html"

# parada e retomada de tudo
hermes pause      # liga o ESTOP (cron/kanban/gateway)
hermes resume     # religa
hermes cron list  # estado dos 7 jobs
```

Ambiente no momento da parada: Arch Linux + Omarchy (7.2.3-arch1-3), Python 3.14.7, `.venv` do projeto com as dependências do motor, `uv` para o venv descartável dos gráficos, Chromium e pandoc para o PDF.

13. Índice de arquivos-chave (para o replanejamento)

~/Code/Coding_Machine/	
├─ README.md	visão e fluxo
├─ Pendencias.md	o que está pendente + mapa dos produtos
├─ LICOES-APRENDIDAS.md	revisão de processo de 29/09 (com nota de correção)
├─ TesteAB-eficiencia-LLM.md	plano do estudo A/B (não executado)
├─ RELATORIO-DE-PARADA.md	este documento
├─ REPORT-TENTATIVAS-CODEX.md	relatório de tentativas (fonte do PDF)
├─ autodev/	21 módulos, 6.281 linhas (o motor)
├─ .autodev/	
│ └─ config/	agents.yaml, models.yaml, policies.yaml
│ └─ tests/	34 arquivos, 450 testes
│ └─ scripts/	tela, verificar_plano, checar_dono_arquivo, estado_00
│ └─ fixtures/	fixtures de teste
│ └─ sprints/DEVFACTORY-00X/	spec, plan/dag, sprint.yaml, decisions, report, HAQ,
│ └─ state.db	fonte da verdade
└─ report/	gerador + PNGs + PDF + HTML + CSS + dados.json
~/workspace/s_llm-coding-efficiency/previa-custo/	medições do A/B (custos.py, medir.py)
~/spec_solucoes_dev/001-vigia-execucao-noturna.md	spec do vigia de execução noturna